```
                    UPDATE muse_quota_ledger
                    SET total_tokens_used = total_tokens_used + ?,
                        total_campaigns_planned =
total_campaigns_planned + 1,
                        updated_at = ?
                    WHERE id = 1 AND (total_tokens_used + ?) <= ?
                    RETURNING total_tokens_used;
                    """,
                    (
                        estimated_tokens,
                        now_ts,
                        estimated_tokens,
                        cls.SURFING_CUTOFF_98_PERCENT,
                    ),
                )
                row = await cur.fetchone()
                if not row:
                    await conn.execute(
                        "UPDATE muse_quota_ledger SET
active_engine_mode = '9ROUTER_ROLLING', updated_at = ? WHERE id = 1;",
                        (now_ts,),
                    )
                    return False
                return True

            reserved = await DB_WRITER_DAEMON.execute_write_tx(db,
_reserve_tx)
            if reserved:
                return "MUSE_PRIMARY", "meta-muse-spark"

        return "9ROUTER_ROLLING", "gemini-4-argon"


#
================================================================================
========
# 6. HARD-FLOOR MARGIN GUARD (80-95% GUARANTEED) & SPLIT-SETTLEMENT
HARVESTER
# Mencegah kebocoran margin Tier Ultra di Zona 3 & melindungi Working
Capital
#
================================================================================
========
PPP_EFFECTIVE_COIN_IDR = {
    1: 580,
    2: 260,
    3: 84,
}
```

```
def calculate_dynamic_safe_coin_cost(
    base_coins: int,
    estimated_hpp_idr: int,
    ppp_zone: int = 3,
    target_min_net_margin: float = 0.82,
    gateway_fee_ratio: float = 0.03,
) -> int:
    """
    Menjamin setiap pesanan berbayar menghasilkan minimal 82% net
margin
    setelah dikurangi HPP GPU/API dan potongan payment gateway.
    Jika HPP Rp 0 (Upscale 4K / Sandbox), otomatis mengembalikan
base_coins (100% Margin).
    """
    if estimated_hpp_idr <= 0:
        return base_coins
    coin_val_idr = PPP_EFFECTIVE_COIN_IDR.get(ppp_zone, 84)
    allowed_hpp_share = max(0.05, 1.0 - target_min_net_margin -
gateway_fee_ratio)
    min_required_revenue_idr = estimated_hpp_idr / allowed_hpp_share
    min_safe_coins = math.ceil(min_required_revenue_idr /
coin_val_idr)
    return max(base_coins, min_safe_coins)


async def record_job_and_split_profit(
    db: aiosqlite.Connection,
    revenue_idr: int,
    actual_hpp_idr: int,
    buffer_ratio: float = 0.10,
) -> dict:
    reserved_working_capital = min(
        revenue_idr, int(math.ceil(actual_hpp_idr * (1.0 +
buffer_ratio)))
    )
    net_harvestable_profit = max(0, revenue_idr -
reserved_working_capital)

    async def _tx(conn: aiosqlite.Connection) -> None:
        now_ts = int(time.time())
        await conn.execute(
            """
            INSERT OR IGNORE INTO treasury_summary (
                id, gross_revenue_idr, working_capital_vault_idr,
                harvestable_profit_vault_idr,
total_harvested_by_owner_idr, updated_at
```

```
            ) VALUES (1, 0, 0, 0, 0, ?);
            """,
            (now_ts,),
        )
        await conn.execute(
            """
            UPDATE treasury_summary
            SET gross_revenue_idr = gross_revenue_idr + ?,
                working_capital_vault_idr = working_capital_vault_idr
+ ?,
                harvestable_profit_vault_idr =
harvestable_profit_vault_idr + ?,
                updated_at = ?
            WHERE id = 1;
            """,
            (
                revenue_idr,
                reserved_working_capital,
                net_harvestable_profit,
                now_ts,
            ),
        )

    await DB_WRITER_DAEMON.execute_write_tx(db, _tx)
    return {
        "working_capital_locked_idr": reserved_working_capital,
        "net_profit_ready_to_harvest_idr": net_harvestable_profit,
        "achieved_net_margin_pct": round(
            (net_harvestable_profit / revenue_idr) * 100.0, 2
        )
        if revenue_idr > 0
        else 100.0,
    }
```

Konfigurasi LiteLLM + 9Router Self-Healing Proxy (litellm_zero_ops.yaml)

Berkas konfigurasi berikut dirancang untuk dijalankan berdampingan dengan 9Router pada Tahap 2 guna mengeliminasi *Cooldown Landmines*, mengaktifkan pemeriksaan kesehatan latar belakang yang mengabaikan galat transien (*HTTP 429/408*), dan mengisolasi kuota per-agen secara otomatis:

```
model_list:
  - model_name: gemini-4-argon
    litellm_params:
      model: openai/gemini-4-argon
      api_base: http://localhost:20128/v1
```

```
      api_key: os.environ/NINE_ROUTER_KEY
      rpm: 120
      tpm: 250000
      max_tokens: 2048
      order: 1

  - model_name: gemini-4-argon
    litellm_params:
      model: gemini/gemini-2.5-pro
      api_key: os.environ/GOOGLE_AI_STUDIO_KEY
      rpm: 60
      tpm: 200000
      order: 2

  - model_name: nousresearch/hermes-3-llama-3.1-405b
    litellm_params:
      model: openrouter/nousresearch/hermes-3-llama-3.1-405b
      api_key: os.environ/OPENROUTER_API_KEY
      rpm: 60
      tpm: 150000
      max_tokens: 1024

general_settings:
  background_health_checks: true
  health_check_interval: 30
  enable_health_check_routing: true
  health_check_ignore_transient_errors: true

router_settings:
  routing_strategy: latency-based-routing
  num_retries: 2
  cooldown_time: 60
  enable_pre_call_checks: true
  allowed_fails_policy:
    AuthenticationErrorAllowedFails: 1
    TimeoutErrorAllowedFails: 3
    RateLimitErrorAllowedFails: 5
```

Kesimpulan dan Protokol Eksekusi Otonom

Hasil audit komprehensif terhadap Master Blueprint v2.5 menegaskan bahwa desain makro **bikinAI Super Intelligence** telah memiliki seluruh prasyarat fundamental untuk beroperasi sebagai mesin bisnis AI multimodal global di bawah kendali satu operator tunggal. Melalui penyempurnaan *additive* ke spesifikasi **v2.5.1 (Zero-Ops Edition)**, ketujuh celah tersembunyi yang berisiko memicu interupsi operasional tengah malam telah dimitigasi secara struktural di tingkat kode dan arsitektur. Penggantian akses tulis konkuren pada aiosqlite dengan

SingleWriterSQLiteDaemon berbasis transaksi eksplisit BEGIN IMMEDIATE, dikombinasikan dengan pembersihan otomatis tabel phash_audit_log dan pemangkasan berkala berkas jurnal .db-wal (PRAGMA wal_checkpoint(PASSIVE)), menjamin server resepsionis 24/7 tidak akan pernah mengalami kegagalan SQLITE_BUSY ataupun kehabisan ruang disk seiring bertambahnya usia sistem.
Pada lapisan komputasi media dan kecerdasan kognitif, penerapan manajemen memori **Stage-Gated /dev/shm**, integrasi pengunci wajah ringan **ComfyUI-PuLID-Flux2 (Flux.2 Klein 4B/9B)**, penambahan model video cepat **Wan 2.2 5B MoE** serta **LTX-2.3 Distilled**, dan peningkatan mesin suara ke **Kokoro-FastAPI** bersama **Fish Audio S2** menghilangkan sepenuhnya risiko *crash* VRAM 34 GB pada kartu GPU 24 GB. Di saat yang sama, penyisipan proksi **LiteLLM** dengan kebijakan health_check_ignore_tran[span_137](start_span)[span_137](end_span)[span_139](start_span)[span_139](end_span)sient_errors: true dan pembatasan kunci virtual per-agen di depan 9Router melindungi ketujuh agen LangGraph dari efek berantai *Cooldown Landmine*.
Dari perspektif keamanan siber dan kepastian finansial, penyematan **Local Ephemeral Cryptographic Nullifier** pada redeem_stateless_ed25519_voucher_once menutup rapat celah penggandaan voucher di Studio Web Independen NocturneAI.art tanpa melanggar prinsip *Zero-DB Legal Firewall*, sementara algoritma *Sliding-Window Anagram De-Obfuscator* menggagalkan teknik penyelundupan "ilja kaerb" dan substitusi *leetspeak* pada milidetik pertama. Akhirnya, penerapan rumus pelindung margin **calculate_dyn[span_42](start_span)[span_42](end_span)amic_saf[span_45](start_span)[span_45](end_span)e_coin_cost** bersama pemotongan dompet berjenjang (*Waterfall Dual-Wallet*) mengunci margin laba bersih setiap transaksi berbayar di atas 82% hingga 97% di seluruh zona PPP, yang diperkuat oleh arus kas bermargin 100% hingga >100% dari pipa *Zero-Cost 4K Upscale*, *Preset Marketplace*, dan arbitrase *Cliffhanger Blur*. Dengan telah siapnya 17 kredensial inti tanpa kartu Visa serta solusi pintas konsol CLI Vast.ai dan lingkungan sementara GitHub Codespaces, eksekusi konstruksi Tahap 2 dan Tahap 3 dapat langsung dijalankan secara penuh tanpa hambatan teknis maupun finansial.

Karya yang dikutip

1. Health Check Driven Routing | liteLLM, https://docs.litellm.ai/docs/proxy/health_check_routing 2. LiteLLM Rate Limits: RPM, TPM, and Gateway Fixes - Fastio, https://fast.io/resources/litellm-rate-limit/ 3. Best GPU for AI video: What the benchmarks show in 2026 - Runpod, https://www.runpod.io/articles/guides/best-gpu-for-ai-video 4. PuLID for Flux: Identity Injection Without Training, https://iimagined.ai/blog/pulid-flux 5. SQLite async connection pool for high-performance - Hacker News, https://news.ycombinator.com/item?id=44530518 6. Why I'm using SQLite as the only database for a production SaaS, https://www.reddit.com/r/FastAPI/comments/1ra1xlg/why_im_using_sqlite_as_the_only_database_for_a/ 7. Deploy Kokoro TTS on Dokploy, https://dokploy.com/templates/kokoro-tts 8. Cross-Tenant Interference Attacks on LLM Gateways - arXiv, https://arxiv.org/pdf/2610.05089 9. Fallbacks (Provider Failover) | liteLLM, https://docs.litellm.ai/docs/proxy/reliability 10. Router - Load Balancing | liteLLM, https://docs.litellm.ai/docs/routing 11. Proxy - Load Balancing | liteLLM, https://docs.litellm.ai/docs/proxy/load_balancing 12. Mastering Cinematic Video with WAN 2.2 in Comfy UI - Civitai, https://civitai.com/articles/17844/mastering-cinematic-video-with-wan-22-in-comfy-ui 13. How to Run Wan 2.2 Image to Video Model in ComfyUI (FP8 Version), https://www.nextdiffusion.ai/tutorials/exploring-the-new-wan22-image-to-video-generation-model

-in-comfyui 14. WAN 2.1 vs 2.2 vs 2.6: The Complete Version Comparison, https://neurocanvas.net/blog/wan-version-comparison/ 15. Wan 2.1 Models | ComfyUI_examples - GitHub Pages, https://comfyanonymous.github.io/ComfyUI_examples/wan/ 16. Face Cloning in ComfyUI | IPAdapter-Plus, ReActor, PuLID-Flux, https://www.youtube.com/watch?v=2nJXjLvirW8 17. Deploy Open-Source TTS on GPU Cloud: Kokoro, Fish Speech, and, https://www.spheron.network/blog/deploy-open-source-tts-gpu-cloud-2026/ 18. First PuLID for FLUX.2 Klein (4B/9B) : r/comfyui - Reddit, https://www.reddit.com/r/comfyui/comments/1rtyafb/release_comfyuipulidflux2_first_pulid_for_flux2/ 19. README.md - aadebuger/ComfyUI-AgeShift - GitHub, https://github.com/aadebuger/ComfyUI-AgeShift/blob/main/README.md 20. TeaCache - ComfyUI Cloud, https://comfy.icu/node/TeaCache 21. ANIMA_BOOSTER detailed guide | ComfyUI - RunComfy, https://www.runcomfy.com/comfyui-nodes/ANIMA_BOOSTER 22. Help with ComfyUI + WAN 2.1 workflow (Triton, Sage, TeachAce, https://www.reddit.com/r/comfyui/comments/1kx3q6z/help_with_comfyui_wan_21_workflow_triton_sage/ 23. BlackSnowSkill/ANIMA_BOOSTER: High-performance ... - GitHub, https://github.com/BlackSnowSkill/ANIMA_BOOSTER 24. Kokoro-FastAPI Using Docker - Open WebUI, https://docs.openwebui.com/features/chat-conversations/audio/text-to-speech/Kokoro-FastAPI-integration/ 25. RegKit — Documentation, https://regkit.dev/docs 26. If you're talking to a 100KB SQLite database file this kind of thing is, https://news.ycombinator.com/item?id=44565652 27. This is strange on so many levels. SQLite does not even do network, https://news.ycombinator.com/item?id=44565326 28. Indexing & Embeddings | Chaos Cypher, https://chaoscypher.com/docs/architecture/extraction-pipeline/indexing 29. How sqlite-vec Works for Storing and Querying Vector Embeddings, https://medium.com/@stephenc211/how-sqlite-vec-works-for-storing-and-querying-vector-embeddings-165adeeeceea

Audit Arsitektur Sistem, Ekspansi Infrastruktur, dan Rekayasa Keamanan Tingkat Lanjut: Ekosistem Otonom bikinAI Super Intelligence

Evaluasi Fondasi Blueprint Saat Ini dan Pemetaan Kesenjangan Arsitektural

Arsitektur yang tertuang dalam 50% dokumen cetak biru (*master blueprint*) **bikinAI Super Intelligence** telah menunjukkan kematangan desain sistem terdistribusi yang sangat solid untuk model bisnis agregator dan arbitrase komputasi AI. Keputusan arsitektural untuk memisahkan secara tegas antara **Kantor Resepsionis** yang beroperasi 24/7 dengan kebutuhan sumber daya mikro (Bot Omnichannel, 9Router, 5 Agen Otonom, dan SQLite WAL pada Oracle Cloud Free Tier Ampere A1 ARM atau VPS KVM Rp 25.000/bulan) dengan **Pabrik Render GPU** berbasis *On-Demand/Serverless* (Vast.ai, RunPod RTX 3090/4090 24 GB VRAM, serta API grosir *pay-per-call* seperti Fal.ai, Replicate, Novita AI, PiAPI, dan WaveSpeed AI) merupakan fondasi ekonomi yang tepat untuk menekan biaya tetap (*fixed cost*) hingga mendekati Rp 0–Rp 41.000 per bulan.

Selain efisiensi infrastruktur dasar, cetak biru tersebut telah berhasil mengintegrasikan strategi akuisisi pasar berbasis *Omnichannel* ("1 Otak, 3 Pintu" melalui Telegram, Facebook Messenger dengan *Auto-Reply Comment*, dan WhatsApp via Cloud API/WAHA), rekayasa *98% Quota Surfing* pada 9Router (decolua/9router) yang dikombinasikan dengan kompresi token RTK & Caveman, psikologi *Endowed Progress Effect* lewat klaim harian Rp 400 (4 Koin Bonus) dengan isolasi dua dompet (koin_utama dan koin_bonus), hingga matriks pertahanan siber 6-lapisan (TMA-01 s/d INFRA-01), mitigasi agregator (AGG-01 s/d AGG-05), dan penambalan celah *Red-Team* (SEC-01 s/d SEC-06) beserta pemisahan entitas hukum melalui *Universal Compute Voucher* (NocturneAI.art / VelvetRender.sh).

Namun, ketika sistem ini bergerak dari tahap prototipe menuju skala produksi komersial global yang melampaui 6.000 video per bulan dan puluhan ribu permintaan teks maupun dokumen harian, evaluasi arsitektural mendalam mengidentifikasi sejumlah celah struktural, risiko *bottleneck* konkurensi, dan peluang efisiensi biaya yang belum tertutup pada 50% dokumen pertama. Tabel berikut merangkum audit silang terhadap seluruh komponen cetak biru saat ini beserta ekspansi aditif yang perlu diimplementasikan untuk mencapai ketahanan tingkat *enterprise*.

Komponen Blueprint Saat Ini	Kekuatan Desain Eksisting	Celah Arsitektural & Risiko Skala Produksi	Rekomendasi Ekspansi Aditif (Solusi Tingkat Enterprise)
Orkestrasi 5 Agen & Muse Router (muse_router.py)	Pembagian tugas yang jelas antara Gatekeeper, Orchestrator, Prompt	Alur eksekusi masih bersifat sekuensial-linier (tanpa *cyclic self-correction*	Transformasi ke **7-Node LangGraph Hierarchical Supervisor** dengan

Komponen Blueprint Saat Ini	Kekuatan Desain Eksisting	Celah Arsitektural & Risiko Skala Produksi	Rekomendasi Ekspansi Aditif (Solusi Tingkat Enterprise)
	Engineer, Media Worker, dan CS Triage dengan biaya modal Rp 0.	saat salah satu klip dari 6 adegan gagal/drift) dan belum memiliki agen khusus pengunci konsistensi wajah/karakter lintas adegan.	penambahan *Visual Continuity Agent* (PuLID/InstantID) dan *V2A Foley & Mastering Agent* (MMAudio + Kokoro-82M).
Dapur 9Router & Manajemen Kuota (localhost:20128/v1)	*98% Quota Surfing*, kompresi token RTK/Caveman (hemat 20–65%), dan *3-Tier Fallback* otomatis ke OpenRouter :free.	Setiap kueri berulang (misal: template kampanye UMKM atau terjemahan prompt serupa) tetap memicu inferensi LLM penuh, membakar kuota harian 98% lebih cepat pada jam sibuk.	Integrasi **Semantic Caching & Vector Routing** (Valkey / RedisVL) di depan 9Router yang memangkas panggilan API hingga 68,8–86% dengan latensi *cache hit* ~25 ms.
Pabrik Render Video & Audio 30 Detik (stateless_gpu_worker.py)	Memecah video 30 detik menjadi 6 klip x 5 detik dan menjahitnya dengan FFmpeg.	*Character drift* (wajah dan pakaian berubah drastis antar klip ke-1 hingga ke-6), video AI bisu tanpa efek suara lingkungan (*foley*) yang sinkron per *frame*, serta *cold-start* GPU.	Implementasi rantai *Last-Frame Extraction* + *PuLID Flux II / Wan 2.2 MoE*, akselerasi **TeaCache + SageAttention + torch.compile** (54% lebih cepat), serta injeksi otomatis **MMAudio + Kokoro-82M**.
Kode Pertahanan Konkurensi & Database (config_and_db.py)	Menggunakan USER_LOCKS = defaultdict(asyncio.Lock) dan UPDATE ... RETURNING pada SQLite WAL untuk mencegah *race condition* TOCTOU (SEC-02).	Struktur defaultdict(asyncio.Lock) tanpa mekanisme *eviction* akan mengalami kebocoran memori (*memory leak*) jika diserang jutaan ID akun bot. Selain itu, SQLite WAL memiliki batasan *single-writer* yang memicu SQLITE_BUSY saat ribuan *webhook* dan klaim harian menulis bersamaan.	Refactoring ke **Fixed-Size Lock Striping**, antrean *Async Write-Batcher* untuk SQLite WAL, dan abstraksi lapisan data menuju **Turso (libSQL) Concurrent Writes** / **PostgreSQL**.
Keamanan Input Multi-Modal &	Sanitasi Unicode NFKC (SEC-01), batas header	Fitur pembaca PDF/Word/Excel rentan	Dekomposisi dokumen di *sandbox* terisolasi

Komponen Blueprint Saat Ini	Kekuatan Desain Eksisting	Celah Arsitektural & Risiko Skala Produksi	Rekomendasi Ekspansi Aditif (Solusi Tingkat Enterprise)
Anti-Tuyul (security_gate.py)	24-byte (SEC-06), InsightFace >23 tahun (AI-SEC-02), dan sistem 2 dompet.	terhadap *Indirect Prompt Injection*. Klaim koin harian Rp 400 dan afiliasi rentan ditambang oleh *emulator farm* (tuyul) yang mengakali diskon Rp 500/video.	dengan **Llama Prompt Guard 2 (86M) + NeMo Guardrails**, serta penyematan **TMA Behavioral Biometrics, WebGL Fingerprinting, & Cloudflare Turnstile**.

Dimensi 1: Ekspansi Stack AI Engine (SOTA) dan Orkestrasi Multi-Agen Hirarkis

Integrasi Aditif Engine SOTA Multi-Modal Berbiaya Ultra-Rendah

Untuk mempertahankan keunggulan kompetitif "Sultan Super-App" sekaligus menekan Harga Pokok Penjualan (HPP) di bawah batas psikologis kompetitor, tumpukan (*stack*) engine AI pada **bikinAI Super Intelligence** perlu diperluas dengan model *State-of-the-Art* (SOTA) generasi terbaru yang memiliki efisiensi *Mixture-of-Experts* (MoE) dan kuantisasi presisi rendah. Penambahan ini bersifat murni aditif terhadap engine yang sudah terdaftar pada cetak biru, yakni Gemini Flash, Qwen 2.5, Llama 3.3, DeepSeek R1/V3, Claude Sonnet, Nous Hermes 3, Mistral Nemo, Flux.1, Wan 2.1/2.2, LTX-Video, Hunyuan, Kling, Veo, Sora, Runway, dan Suno/Udio.

Pada lapisan penalaran teks, orkestrasi, dan multimoda, integrasi **DeepSeek-V4.1-Flash** dan **DeepSeek-V4-Pro** (melalui SiliconFlow, OpenRouter, atau Parasail dengan dukungan *Zero Data Retention* / ZDR) memberikan lompatan efisiensi yang masif. DeepSeek-V4.1-Flash merupakan model MoE multimodal dengan tulang punggung 552 miliar parameter yang hanya mengaktifkan sekitar 8 miliar parameter saat fase *prefill* dan 16 miliar parameter saat *decode*, menghasilkan kecerdasan setara model *flagship* dengan tarif input serendah $0,042–0,06 per 1 juta token (dan *cached input* $0,01 per 1 juta token) serta output $0,084–0,24 per 1 juta token. Untuk tugas pemrograman kompleks dan agen otonom, **Qwen 3.8 Max**, **Qwen 3.7 Plus** ($0,14 input / $0,55 output per 1 juta token), **Qwen3-Omni-30B-A3B-Thinking**, serta **Kimi K2.7 Code** menjadi tulang punggung tambahan di dalam 9Router ketika kuota gratis harian mencapai ambang rotasi 98%.

Pada lapisan generasi video mandiri (Mode 3: Pabrik GPU ComfyUI) dan grosir (Mode 2: API *Pay-Per-Generate*), sistem wajib mengadopsi arsitektur penuh keluarga **Wan 2.2 MoE (5B & 14B)** serta **Wan 2.5 / 2.6**, berdampingan dengan **Seedance 1.5 Pro** (BytePlus, dengan tarif turun drastis ke $0,012 per detik render), **Open-Sora 2.0 (11B)**, dan **MiniMax H3**. Keunggulan krusial Wan 2.2 terletak pada pemisahan pakar (*expert*) untuk tahap *high-noise* dan *low-noise* yang menghasilkan detail visual jauh lebih tajam tanpa penalti komputasi tambahan, sementara rilis Wan 2.5 mempercepat render 720p hingga 30%.

Celah terbesar pada pembuatan video AI 30 detik di pasar saat ini adalah ketiadaan narasi suara berlatensi instan yang murah serta ketiadaan efek suara lingkungan (*foley*) yang menyatu dengan gerakan visual. Menggunakan Suno v4/v6 (yang memiliki HPP lebih tinggi dan dirancang untuk komposisi lagu utuh di Tier Ultra) atau ElevenLabs ($18–80 per 1 juta karakter)

untuk sekadar mengisi narasi iklan UMKM 30 detik akan menggerus margin keuntungan. Untuk mengatasi hal tersebut, arsitektur perlu mengintegrasikan **Kokoro-82M (Hexgrad)** sebagai mesin *Text-to-Speech* (TTS) utama dan **MMAudio (157M)** bersama **HunyuanVideo-Foley** sebagai mesin *Video-to-Audio* (V2A). Kokoro-82M adalah model *open-weight* berlisensi Apache 2.0 dengan 82 juta parameter yang mencapai latensi audio pertama (*Time-to-First-Byte*) terendah di industri sebesar 28 ms pada GPU modern (skor ELO 1.059). Jika diakses melalui API OpenRouter atau penyedia terkelola, biayanya hanya $0,62 per 1 juta karakter (sekitar Rp 1 per naskah video 30 detik), dan model ini cukup ringan untuk dijalankan langsung di RAM worker tanpa membebani VRAM utama. Sementara itu, MMAudio dan HunyuanVideo-Foley bekerja menggunakan algoritma *flow-matching* dan penyelarasan tingkat *frame* (melalui ekstraktor fitur CLIP pada 8 FPS dan Synchformer pada 25 FPS) untuk membaca klip video bisu hasil render Wan atau Flux, lalu mensintesis efek suara langkah kaki, pergerakan objek, maupun ambien ruangan beresolusi 48kHz yang tersinkronisasi presisi dengan visual hanya dalam waktu 1,23 detik untuk klip berdurasi 8 detik.

Untuk mengatasi masalah *character drift*—yakni perubahan struktur wajah, gaya rambut, dan pakaian karakter saat Muse memecah video 30 detik menjadi 6 klip berdurasi 5 detik—tumpukan visual wajib menambahkan **PuLID Flux II**, **FLUX.2** (yang mendukung hingga 10 gambar referensi sekaligus), serta kombinasi **InstantID + IP-Adapter** di dalam alur kerja ComfyUI. Berbeda dengan *InstantID* standar yang terkadang mengalami penurunan kemiripan wajah saat karakter menoleh ke samping (*profile view*), *PuLID Flux II* mengunci identitas wajah secara konsisten tanpa mencemari bobot gaya model utama (*zero model pollution*).

Kategori Modalitas	Engine AI Tambahan (Aditif)	Lisensi / Akses	Spesifikasi & Keunggulan Teknis	Biaya Operasional (HPP)	Penempatan Kasta (Tier)
LLM & Reasoning	**DeepSeek-V4.1-Flash** (552B MoE, 8B/16B aktif)	Open-Weight / API (SiliconFlow, OpenRouter)	Konteks 1,05M token, arsitektur *Causal Encoder-Decoder*, mendukung ZDR.	$0,042–0,06 / 1M input; $0,01 *cached*	Tier Eco & Otak Muse (Cadangan Lapis 2)
LLM Multimodal	**Qwen 3.7 Plus / Qwen3-Omni-30B-A3B**	Open-Weight / API	Pemahaman dokumen, audio, dan instruksi multibahasa Asia Tenggara yang presisi.	$0,14 / 1M input; $0,55 / 1M output	Tier Pro (Otak Kampanye & Analisis PDF)
Video Generatif (Self-Hosted)	**Wan 2.2 MoE (5B & 14B FP8)** & **Wan 2.5**	Apache 2.0 (Komersial Penuh)	Arsitektur pakar *high-noise/low-noise*, kualitas 99,5% setara FP16 dengan separuh VRAM (12–14 GB).	Rp 110–Rp 250 / klip 5 detik (di RTX 4090)	Tier Pro (Mesin Utama Margin 85%)
Video Komersial	**Seedance 1.5 Pro** &	API BytePlus / Fal.ai /	Gerakan tarian/aksi	~$0,012 / detik (Rp 950 / klip 5	Tier Pro+ / Tier Ultra

Kategori Modalitas	Engine AI Tambahan (Aditif)	Lisensi / Akses	Spesifikasi & Keunggulan Teknis	Biaya Operasional (HPP)	Penempatan Kasta (Tier)
(API Grosir)	**Open-Sora 2.0 (11B)**	ComfyUI	sangat mulus, kepatuhan prompt sinematik tinggi.	detik)	
Konsistensi Karakter	**PuLID Flux II** & **FLUX.2 Multi-Ref**	Open-Source Node ComfyUI	Mengunci identitas wajah & pakaian lintas 6 klip video (30 detik) tanpa *model pollution*.	Rp 0 (Berjalan di dalam pipeline GPU yang sama)	Tier Pro & Paket Story 30 Detik
Voiceover / TTS Instan	**Kokoro-82M v1.0**	Apache 2.0 (OpenRouter / Lokal)	Latensi 28 ms TTFB, kualitas suara natural (ELO 1.059), ukuran model sangat ringan.	$0,62 / 1M karakter (~Rp 1 / narasi iklan)	Tier Eco (Gratis) & Tier Pro (Narasi Video)
Efek Suara Video (V2A Foley)	**MMAudio (157M)** & **HunyuanVideo-Foley**	Open-Source (CVPR 2025 / Tencent)	Menghasilkan audio foley & ambien 48kHz yang sinkron per *frame* dalam 1,23 detik.	~Rp 15 / klip 8 detik (Inferensi GPU 1,2 detik)	Paket Story 30 Detik & Tier Ultra

Evolusi 5 Agen Menjadi 7-Node LangGraph Hierarchical Supervisor

Dalam cetak biru 50% saat ini, kelima agen (*Gatekeeper & Billing*, *Orchestrator/Muse*, *Prompt Engineer & Sanitizer*, *Media Worker & Compiler*, serta *AI Support & Triage*) dihubungkan melalui pemanggilan skrip Python asinkron standar. Ketika pengguna mengeksekusi fitur unggulan **"1 Perintah Jadi Paket Kampanye Utuh"** atau **"Paket Story / Iklan Utuh 30 Detik (6 Klip + Jahit FFmpeg + Voiceover/Musik)"** seharga 100 Koin (Rp 10.000), eksekusi linier menghadapi risiko kegagalan berantai: apabila klip ke-4 mengalami distorsi visual atau wajah karakter berubah, sistem linier tetap menjahitnya menjadi video cacat yang memicu komplain dan *refund* penuh.

Solusi arsitektural tingkat lanjut adalah memformalkan orkestrasi menggunakan **LangGraph Hierarchical Supervisor Pattern** (StateGraph) di mana **Muse Orchestrator** bertindak sebagai *Central Traffic Controller* yang memegang kendali atas objek status bersama (*Shared Workflow State*) menggunakan primitif Command(goto=...) dan anotasi Literal[...]. Pekerja spesialis (*worker agents*) tidak pernah memutuskan sendiri agen mana yang berjalan selanjutnya; mereka mengeksekusi tugas spesifik, memperbarui *graph state*, dan mengembalikan kontrol ke Supervisor untuk dievaluasi sebelum alur kerja dilanjutkan atau diinstruksikan melakukan *self-correction loop*. Selain mempertahankan 5 agen asli secara utuh, arsitektur ini diperluas menjadi **7 Agen Spesialis** dengan menyematkan **Visual Continuity & Character Lock Agent**

(Agen #6) serta **V2A Foley & Audio Mastering Agent** (Agen #7).
Di dalam siklus eksekusi *StateGraph* tersebut, *Visual Continuity & Character Lock Agent* bertugas mengekstrak *frame* terakhir dari klip ke-N menggunakan FFmpeg, membangun *turnaround sheet* (referensi multi-sudut tampak depan, 3/4, dan profil samping), serta menyuntikkan *embedding* identitas wajah melalui *PuLID Flux II* beserta penguncian deskripsi pakaian ke dalam *start frame* klip ke-(N+1) sehingga transisi antar-adegan 30 detik bebas dari *character drift*. Selanjutnya, setelah keenam klip video selesai dirender, *V2A Foley & Audio Mastering Agent* mengambil alih untuk memisahkan produksi suara menjadi tiga jalur (*stems*) independen: narasi suara (*voiceover*) dari Kokoro-82M, lapisan suara ambien dan efek gerakan sinkron dari MMAudio/HunyuanVideo-Foley, serta musik latar dari Suno/Udio. Agen ke-7 ini secara otomatis menerapkan *audio ducking* (menurunkan desibel ambien dan musik saat narasi berbicara), melakukan *crossfade* pada sambungan antar-segmen, dan menormalisasi kekerasan audio sebelum menyerahkan aset akhir ke *Media Worker & Compiler Agent* untuk dijahit menjadi satu file .mp4 utuh.
Secara makro, topologi sistem yang telah diperluas ini beroperasi dalam empat zona arsitektur yang terisolasi secara ketat. Zona Pertama (*Omnichannel Ingress & Gatekeeper*) menangani pesan masuk dari Telegram, WhatsApp, dan Facebook Messenger melalui pemotongan koin *escrow* atomik dan gerbang keamanan berlapis. Zona Kedua (*24/7 Receptionist VPS*) menjalankan *LangGraph 7-Agent Supervisor* di atas mesin distribusi 9Router dan lapisan *Valkey Semantic Cache*. Zona Ketiga (*Dual Execution Pipelines*) membelah eksekusi beban berat menjadi Jalur Komersial SFW (Fal.ai, Replicate, RunPod Wan 2.2, Suno) dan Jalur Sandbox Terisolasi (*Universal Compute Voucher* pada domain independen dengan pemrosesan RAM /dev/shm). Terakhir, Zona Keempat (*Zero-Knowledge Edge Delivery*) memanfaatkan *Cloudflare Workers Relay* untuk menyedot media dari hulu, melucuti metadata EXIF secara *streaming*, mengenkripsi file menjadi biner .bin di penyimpanan Cloudflare R2, dan mendekripsinya langsung di memori RAM peramban pengguna menggunakan *Web Crypto API*.

Pipeline Routing Dinamis dan Arsitektur Isolasi Kontainer untuk Model Unfiltered

Sesuai dengan visi produk pada config.py (PILIHAN_OTAK_UNFILTERED yang mencakup **Nous Hermes 3 Uncensored**, **Qwen 2.5 72B Instruct**, dan **Mistral Nemo Unfiltered**, serta kapabilitas render visual tanpa sensor di Novita AI dan ComfyUI mandiri), sistem dirancang untuk melayani kebebasan berekspresi pengguna global (termasuk genre horor, fiksi dewasa, dan eksperimen artistik tanpa sensor) sembari tetap menegakkan pencegahan mutlak terhadap konten ilegal internasional (seperti CSAM, kekerasan dunia nyata, atau *deepfake* non-konsensual terhadap tokoh nyata yang dicegat oleh *InsightFace >23 Tahun* dan *Negative Tensor*). Namun, mengizinkan lalu lintas *unfiltered* bercampur di dalam proses memori atau identitas jaringan yang sama dengan Bot Telegram utama (@bikinAI_bot) akan memicu pemblokiran oleh pemindai otomatis Telegram maupun pencabutan akun oleh *payment gateway* QRIS lokal.
Untuk menyempurnakan mekanisme **Pre-Flight Heat Router (AGG-04)** dan **Website Studio Independen (NocturneAI.art / VelvetRender.sh)** yang telah dirancang pada Bagian V cetak biru, *Prompt Engineer & Sanitizer Agent* menerapkan pemetaan **Multi-Dimensional Semantic Heat Scoring** yang membagi setiap permintaan ke dalam empat zona rute eksekusi sebagaimana dirinci pada tabel berikut.

Zona & Rentang Heat Score	Klasifikasi Konteks Semantik	Tindakan Pre-Flight & Transformasi Prompt	Rute Eksekusi Engine & Isolasi Pengiriman
Zona Hijau (Skor 0–25)	Konten komersial UMKM, iklan produk, lanskap, edukasi, animasi umum (100% SFW).	*Zero-Passthrough Canonical Reconstruction* + Injeksi *Forced Negative Prompt*.	Diteruskan langsung ke pipa @bikinAI_bot (Fal.ai, Replicate, Kling, Veo, Wan 2.2) dengan safety_checker: true.
Zona Kuning (Skor 26–60)	Konten ambigu, sinematik gelap, aksi laga/horor fiksi, atau mode busana yang berisiko memicu *false-positive sensor* di API hulu.	**Muse Auto-Rewrite (AGG-01)** menulis ulang kalimat menggunakan Qwen 2.5 72B / Llama 3.3 70B menjadi narasi artistik (>25 tahun) yang sopan dan aman bagi API hulu.	Dirender pada engine Tier Pro/Ultra yang mendukung kontrol estetika sinematik + pemeriksaan *1ms Shannon Entropy* untuk mencegah layar hitam (*black image*).
Zona Magenta (Skor 61–99)	Permintaan kreatif eksplisit / *Unfiltered* dewasa yang sah (>23 tahun, karakter fiksi murni).	Di ruang chat @bikinAI_bot, bot menolak secara sopan (Rp 0 dipotong) dan menyajikan tombol konversi **[Tukar Koin ke Compute Voucher]**.	Dieksekusi secara eksklusif di Studio Web Independen (NocturneAI.art / VelvetRender.sh) via **Nous Hermes 3 Uncensored** & GPU Worker /dev/shm (enable_safety_checker: false) + pengiriman .bin AES-GCM.
Zona Merah Fatal (Skor 100)	Indikasi CSAM, unggahan wajah usia <23 tahun pada *InsightFace*, *deepfake* selebriti/tokoh nyata, atau instruksi bahaya fisik.	**Hard-Block di Gerbang Lokal pada Rp 0** sebelum menyentuh Master API Key mana pun.	Permintaan dihentikan total, dicatat ke log keamanan internal, dan akun dikenakan *cooldown* otomatis tanpa merampas saldo uang yang memicu sengketa bank.

Dalam rancangan cetak biru saat ini, pengguna menukar koin menjadi kode VOUCHER-XXXX-YYYY untuk dimasukkan ke NocturneAI.art. Apabila server NocturneAI.art melakukan koneksi database langsung (melalui kueri SQL atau panggilan REST API) ke VPS @bikinAI_bot untuk memvalidasi kode voucher tersebut, jejak paket jaringan (*network flow*) antara kedua server akan menghubungkan kedua entitas secara forensik. Untuk menciptakan **Isolasi Kriptografis Tanpa Koneksi Database (*Zero-Database Link*)**, mekanisme penukaran wajib menggunakan **Stateless Ed25519 Signed Capability Voucher**. Saat pengguna menukar 50 Koin di @bikinAI_bot, bot memotong koin secara atomik dan menandatangani struktur payload biner berisi (voucher_nonce, coin_amount, origin_ppp_tier, expiry_timestamp) menggunakan **Kunci Privat Ed25519** yang hanya ada di server @bikinAI_bot, menghasilkan token kompak VCH-<base64url_payload>.<ed25519_signature>. Server NocturneAI.art hanya menyimpan **Kunci Publik Ed25519** (yang bersifat satu arah dan aman dipublikasikan) untuk

memverifikasi keaslian voucher secara matematis dalam 0,1 milidetik, kemudian mencatat voucher_nonce yang telah diklaim ke dalam database lokalnya sendiri guna mencegah *double-spending*. Dengan arsitektur asimetris ini, tidak ada satu pun koneksi jaringan, panggilan API, atau berbagi database antara @bikinAI_bot (entitas SaaS produktivitas Indonesia) dan NocturneAI.art (platform eksperimen open-source global).
Pada tingkat perangkat keras, ketika worker GPU di Vast.ai atau RunPod mengeksekusi beban kerja dari jalur *Unfiltered*, kontainer Docker wajib dijalankan menggunakan runtime isolasi kernel (seperti runsc / gVisor atau bubblewrap untuk sub-proses) dengan kebijakan jaringan *Egress Whitelist* yang hanya mengizinkan lalu lintas keluar menuju *Cloudflare Worker Edge Relay*. Seluruh bobot model LoRA tambahan, gambar input, dan tensor *latent* diproses secara eksklusif di dalam partisi memori volatil /dev/shm (berbasis tmpfs RAM Linux yang di-mount dengan parameter keamanan noexec,nosuid,nodev,size=16g) serta disuntikkan *Negative Embedding Permanen* pada level *tensor conditioning* GPU. Begitu output selesai dienkripsi menggunakan **AES-256-GCM** menjadi blob .bin di dalam RAM dan dikirim ke *Edge Relay*, ruang memori /dev/shm langsung ditimpa nol (*zero-overwrite*) dan kunci enkripsi sesi dihancurkan tanpa pernah menyentuh piringan diska (*hardisk* fisik host).

Dimensi 2: Hardening Keamanan Berlapis (Defense-in-Depth) dan Mitigasi Anti-Fraud

Penambalan Celah Kritis pada Kode Pertahanan Saat Ini (Audit SEC-01 s/d SEC-06)

Kode pertahanan inti yang terdapat pada Bagian 5.4 dokumen cetak biru telah dirancang secara cerdas untuk menangkis serangan *Financial Race Condition* (TOCTOU SEC-02), *Polyglot Prompt* (SEC-01), dan *Webhook Spoofing* (SEC-03) tanpa ketergantungan pada server Redis eksternal yang boros memori. Meskipun demikian, bedah kode tingkat produksi mengungkap empat celah kerentanan turunan (*second-order vulnerabilities*) yang perlu segera ditambal agar sistem tahan terhadap serangan berskala besar.
Kerentanan pertama terletak pada potensi **Memory Exhaustion (OOM)** akibat deklarasi USER_LOCKS = defaultdict(asyncio.Lock) yang bersifat tak terbatas (*unbounded dictionary*). Walaupun satu instans objek asyncio.Lock hanya memakan sekitar 64 bytes, entri kamus Python beserta referensi objek integer di dalamnya mengonsumsi sekitar 240–280 bytes per kunci unik. Apabila penyerang melancarkan serangan enumerasi ID melalui *webhook* atau *botnet* yang memicu inisialisasi 5 hingga 10 juta user_id acak yang berbeda, USER_LOCKS akan tumbuh secara permanen di memori karena tidak memiliki mekanisme penghapusan (*eviction*), menguras >2 GB RAM dan menjatuhkan VPS kecil. Solusi yang jauh lebih tangguh adalah mengganti defaultdict dengan **Fixed-Size Lock Striping**—yakni mengalokasikan sebuah array statis berisi 4.096 objek asyncio.Lock saat aplikasi menyala, lalu memetakan setiap pengguna menggunakan operasi modulo USER_LOCK_STRIPES[user_id % 4096]. Pendekatan ini mengunci penggunaan RAM di angka ~250 KB selamanya (100% kebal terhadap serangan jutaan ID palsu) sambil tetap menjamin penguncian eksklusif per pengguna.
Kerentanan kedua muncul dari fitur **Auto-Intent Detection untuk Pembacaan Dokumen PDF/Word/Excel** (Bagian 4.1 cetak biru) yang membuka vektor serangan **Indirect Prompt Injection**. Fungsi sanitize_polyglot_prompt saat ini membersihkan karakter tak terlihat Unicode (Cf, Cc, Cs), namun belum mengisolasi teks yang diekstrak dari dokumen eksternal. Penyerang

dapat mengunggah file PDF katalog produk yang di dalamnya telah disisipi instruksi tersembunyi (teks putih berukuran mikro) yang memerintahkan agen untuk memanipulasi status Job_ID atau membocorkan konfigurasi sistem. Untuk memitigasi risiko ini, arsitektur wajib menerapkan **Privilege Separation (Dual-LLM Sandbox)**: pemrosesan teks dokumen PDF/Word/Excel harus ditangani oleh instans LLM pembaca yang sama sekali tidak memiliki akses ke alat (*tools*), fungsi *database*, maupun rute safe_one_time_refund, serta membungkus seluruh teks ekstraksi di dalam tag pembatas XML acak per-sesi.
Kerentanan ketiga adalah **Multi-Turn Context Drift** pada fitur *Smart Context Compression* di dapur 9Router (Bagian 3.2 cetak biru). Penyelarasan keamanan LLM umumnya dioptimalkan untuk menolak instruksi berbahaya pada satu giliran pesan tunggal (*single-turn*), namun rentan ditembus ketika penyerang membangun konteks percakapan yang tampak wajar selama 10–15 giliran lalu mengarahkan model ke topik terlarang. Ketika 9Router merangkum obrolan panjang tersebut menjadi memori padat (*compressed context*), instruksi manipulatif penyerang dapat ikut terkompresi ke dalam *system summary* sehingga dianggap sebagai instruksi terpercaya pada giliran berikutnya. Oleh karena itu, setiap hasil kompresi konteks wajib melewati pemindaian klasifikator keamanan tingkat dialog sebelum disimpan kembali ke memori sesi.
Kerentanan keempat berkaitan dengan **Token Replay Attack di Dalam Jendela 60 Detik pada Validasi initData (API-01)**. Cetak biru telah menetapkan pemeriksaan kedaluwarsa auth_date maksimal 60 detik pada tanda tangan HMAC-SHA256 Telegram.WebApp.initData. Namun, di dalam rentang 60 detik sebelum auth_date kedaluwarsa, penyerang yang menyadap string initData melalui *DevTools* masih dapat menembakkan string yang sama ribuan kali secara paralel untuk mencetak ribuan *Short-Lived JWT*. Selain itu, implementasi HMAC wajib mengecualikan *field* hash dan signature dari string pemeriksaan agar kompatibel dengan pembaruan protokol Ed25519 Telegram terbaru. Menyimpan nilai hash yang sudah ditukar di dalam struktur *Bounded Ring Buffer* berkapasitas tetap selama 65 detik secara efektif menutup celah *replay* tersebut dengan konsumsi RAM di bawah 1 MB.

Arsitektur AI WAF dan Guardrail Tiga Lapis Berlatensi Rendah

Menjalankan LLM berukuran besar sebagai hakim keamanan (*LLM-as-a-Judge*) pada setiap pesan yang masuk akan menambah latensi beberapa detik pada jalur kritis dan menggandakan konsumsi token harian. Sebaliknya, hanya mengandalkan ekspresi reguler (*regex*) atau daftar kata kunci statis gagal mendeteksi lima dari enam kategori kegagalan AI yang bersifat semantik—seperti *polite policy violation*, *roleplay framing*, dan *rephrased jailbreaks*. Untuk mencapai keamanan tingkat *enterprise* tanpa mengorbankan kecepatan respons bot, modul security_gate.py perlu mengimplementasikan arsitektur **Three-Stage Low-Latency AI Guardrail** yang menjaga total anggaran latensi di bawah **80 milidetik (p99)**.
Pada tahap pertama (**Sub-Millisecond Static Gate, <1 ms**), sistem mengeksekusi normalisasi Unicode NFKC, penyaringan karakter kontrol, validasi skema struktur Pydantic, dan pemeriksaan *24-Byte Header Guard* untuk gambar. Apabila lolos, pesan masuk ke tahap kedua (**Fast First-Pass Injection Filter, 20–50 ms**) yang ditenagai oleh **Llama Prompt Guard 2 (86M)**—sebuah model klasifikasi berbasis tulang punggung mDeBERTa dengan 86 juta parameter (~0,3B total parameter) yang hanya membutuhkan memori di bawah 1 GB sehingga dapat berjalan sangat cepat di CPU ARM Oracle Cloud (menggunakan kuantisasi ONNX INT8) maupun di irisan tipis GPU untuk menangkap upaya *prompt injection* dan *jailbreak* sebelum menyentuh model utama. Hanya untuk pesan yang ditandai mencurigakan oleh tapis pertama atau yang mengarah ke pembuatan media sensitif, sistem memicu tahap ketiga (**Dialog & Hazard Classification Rails, 20–65 ms**) yang menggabungkan kerangka kerja *open-source*

NVIDIA NeMo Guardrails (menggunakan bahasa definisi alur *Colang* untuk melacak status percakapan multi-giliran dan meredaksi PII) dengan klasifikator **Llama Guard 3 8B** yang dikuantisasi ke INT4 (memangkas kebutuhan VRAM ke 4 GB dan latensi p50 ke ~20 ms tanpa penurunan akurasi klasifikasi pada 8 bahasa sesuai taksonomi bahaya MLCommons).

Biometrik Perilaku, Device Fingerprinting, dan Mitigasi Eksploitasi Tuyul (Sybil Resistance)

Sistem ekonomi mikro **bikinAI Super Intelligence** mengandalkan instrumen akuisisi **Klaim Harian Rp 400 (4 Koin Bonus)** dan **Bonus Referral +2 Koin Bonus**. Meskipun cetak biru telah memisahkan koin_utama dari koin_bonus serta menetapkan bahwa koin_bonus baru terbuka penuh untuk fitur mahal (Video Flagship/Suno) apabila akun tersebut pernah melakukan *Top-Up* minimal 1 kali (Rp 5.000) seumur hidup, struktur insentif ini masih menyisakan dua celah ekonomi *Sybil* (*tuyul*) yang krusial.

Pertama, terdapat celah **Arbitrase Penimbunan Koin Bonus (*Hoarding Attack*)**. Operator peternakan akun (*bot farm*) dapat menjalankan 200 akun Telegram kloningan menggunakan skrip otomatis untuk mengklaim 4 Koin Bonus per hari selama 30 hari berturut-turut, sehingga setiap akun mengumpulkan tabungan 120 Koin Bonus (senilai Rp 12.000). Pada hari ke-30, operator hanya perlu membayar *Top-Up* QRIS sebesar Rp 5.000 pada satu akun untuk membuka pembatasan (*unlock full access*) atas seluruh 120 Koin Bonus tersebut, sehingga memperoleh daya beli komputasi senilai Rp 17.000 (cukup untuk merender 8 video 5 detik atau 11 lagu Suno yang memiliki modal HPP nyata Rp 3.000–Rp 16.500) hanya dengan uang Rp 5.000. Kedua, ratusan akun tuyul yang tidak pernah melakukan *top-up* tetap dapat menguras kuota *free-tier* untuk menghasilkan ribuan gambar Flux HD dan *Upscale 4K* setiap hari guna dijual kembali di luar platform, yang pada akhirnya mempercepat habisnya kuota harian API milik pelanggan asli.

Untuk menutup celah ini secara permanen, proses klaim koin harian yang sudah ditempatkan di dalam *Telegram Mini App (TMA)* pada cetak biru wajib diperkuat dengan matriks pertahanan *Sybil Resistance* dan pembatasan ekonomi sebagaimana dirinci pada tabel berikut.

Lapisan Anti-Fraud (Anti-Tuyul)	Vektor Serangan yang Dicegah	Mekanisme Teknis Implementasi (Biaya Rp 0)	Kebijakan Penindakan Otomatis (Enforcement)
1. Hardware & WebGL Device Fingerprinting di TMA	Peternakan 50–1.000 akun Telegram kloningan di dalam 1 perangkat Android/Emulator (LDPlayer, Nox, atau skrip Python Telethon).	Saat pengguna membuka Mini App untuk klaim harian, JavaScript mengekstrak *hash* gabungan dari WebGL Renderer/Vendor, AudioContext Oscillator, Canvas 2D Entropy, resolusi layar fisik, dan daftar *font* sistem.	Maksimal **2 telegram_user_id per 1 device_fingerprint_hash** yang diizinkan mengklaim koin harian dalam jendela 24 jam. Akun ke-3 pada perangkat yang sama otomatis ditolak klaimnya.
2. Behavioral Biometrics & Sensor Telemetry	Bot pengklik otomatis (*Auto-Clicker* / *Macro*) dan injeksi *POST*	Merekam kurva akselerasi sentuhan (touchstart -> touchend	Jika varians koordinat klik = 0,0 (klik robotik di piksel yang sama

Lapisan Anti-Fraud (Anti-Tuyul)	Vektor Serangan yang Dicegah	Mekanisme Teknis Implementasi (Biaya Rp 0)	Kebijakan Penindakan Otomatis (Enforcement)
	request langsung ke endpoint klaim harian.	pressure/radius variance), mikro-getaran giroskop HP (DeviceOrientationEvent), dan waktu interaksi sejak halaman dimuat.	persis) atau giroskop bernilai null/statis mutlak di perangkat seluler, tandai akun sebagai SUSPECTED_BOT.
3. Cloudflare Turnstile + Aturan Burn-Rate koin_bonus	Penimbunan (*hoarding*) koin_bonus hingga ratusan koin lalu dibuka serentak dengan *top-up* Rp 5.000.	Menyematkan widget tak terlihat **Cloudflare Turnstile** (Gratis 100%) di dalam TMA sebelum tombol klaim aktif, ditambah **Hard-Cap Saldo koin_bonus Maksimal 25 Koin (Rp 2.500)**.	Pengguna tidak bisa menimbun koin_bonus melebihi 25 Koin (setara 1 video 5 detik + 1 gambar HD). Koin klaim baru hanya masuk jika saldo koin_bonus saat ini < 25 Koin, memaksa sirkulasi sehat dan memustahilkan serangan *hoarding* 30 hari.

Implementasi kode produksi berikut mengintegrasikan perbaikan *Fixed-Size Lock Striping* (SEC-02), validasi kriptografi initData dengan *Bounded Replay Guard* (API-01), serta pengikatan *Device Fingerprint* dan batas *Hoarding Cap* koin_bonus:

```
import asyncio
import hmac
import hashlib
import time
from collections import OrderedDict
from urllib.parse import parse_qsl
import aiosqlite

# 1. REFACTOR SEC-02: Fixed-Size Lock Striping (4096 Stripes = ~250KB
RAM Permanen, 0% Memory Leak)
STRIPE_COUNT = 4096
USER_LOCK_STRIPES = [asyncio.Lock() for _ in range(STRIPE_COUNT)]

def get_user_lock(user_id: int) -> asyncio.Lock:
    return USER_LOCK_STRIPES[user_id % STRIPE_COUNT]

# 2. REFACTOR API-01: Bounded Replay Cache untuk initData Hash
(Mencegah Replay dalam Jendela 60 Detik)
class BoundedReplayGuard:
    def __init__(self, max_size: int = 10000, ttl_seconds: int = 65):
        self.cache = OrderedDict()
```

```
        self.max_size = max_size
        self.ttl = ttl_seconds

    def is_replayed_or_register(self, digest_hex: str) -> bool:
        now = time.time()
        while self.cache:
            oldest_key, ts = next(iter(self.cache.items()))
            if now - ts > self.ttl:
                self.cache.popitem(last=False)
            else:
                break
        if digest_hex in self.cache:
            return True
        if len(self.cache) >= self.max_size:
            self.cache.popitem(last=False)
        self.cache[digest_hex] = now
        return False

REPLAY_GUARD = BoundedReplayGuard()

def verify_telegram_webapp_initdata(init_data_raw: str, bot_token:
str, max_age_sec: int = 60) -> dict | None:
    """Verifikasi HMAC-SHA256 initData resmi Telegram + Proteksi
Anti-Replay."""
    parsed = dict(parse_qsl(init_data_raw, keep_blank_values=True))
    received_hash = parsed.pop("hash", None)
    parsed.pop("signature", None)  # Kecualikan field signature sesuai
standar protokol terbaru
    if not received_hash:
        return None

    auth_date = int(parsed.get("auth_date", 0))
    if abs(int(time.time()) - auth_date) > max_age_sec:
        return None

    if REPLAY_GUARD.is_replayed_or_register(received_hash):
        return None

    data_check_string = "\n".join(f"{k}={v}" for k, v in
sorted(parsed.items()))
    secret_key = hmac.new(b"WebAppData", bot_token.encode(),
hashlib.sha256).digest()
    calculated_hash = hmac.new(secret_key, data_check_string.encode(),
hashlib.sha256).hexdigest()

    if not hmac.compare_digest(calculated_hash, received_hash):
        return None
    return parsed
```

```
# 3. ANTI-TUYUL DAILY CLAIM: Device Fingerprint Binding + Max Bonus
Cap (25 Koin)
MAX_KOIN_BONUS_CAP = 25

async def claim_daily_bonus_safe(
    db: aiosqlite.Connection, user_id: int, device_fp_hash: str,
today_str: str
) -> tuple[bool, str]:
    async with get_user_lock(user_id):
        cursor = await db.execute(
            "SELECT COUNT(DISTINCT user_id) FROM daily_claims WHERE
device_fp = ? AND claim_date = ?",
            (device_fp_hash, today_str),
        )
        (device_claim_count,) = await cursor.fetchone()
        if device_claim_count >= 2:
            return False, "DEVICE_LIMIT_EXCEEDED"

        cursor = await db.execute(
            """
            UPDATE users
            SET koin_bonus = MIN(koin_bonus + 4, ?),
                last_claim_date = ?
            WHERE user_id = ?
              AND (last_claim_date IS NULL OR last_claim_date != ?)
              AND koin_bonus < ?
            RETURNING koin_bonus;
            """,
            (MAX_KOIN_BONUS_CAP, today_str, user_id, today_str,
MAX_KOIN_BONUS_CAP),
        )
        row = await cursor.fetchone()
        if not row:
            return False, "ALREADY_CLAIMED_OR_CAP_REACHED"

        await db.execute(
            "INSERT INTO daily_claims (user_id, device_fp, claim_date)
VALUES (?, ?, ?)",
            (user_id, device_fp_hash, today_str),
        )
        await db.commit()
        return True, f"SUCCESS_NEW_BONUS_{row[0]}"
```

Dimensi 3: Refactoring Logika Bisnis, Infrastruktur,

dan Optimasi Efisiensi Biaya (TCO)

Eliminasi Bottleneck Single-Writer SQLite WAL dan Manajemen Memori

Keputusan cetak biru untuk menggunakan **SQLite dalam mode WAL (Write-Ahead Logging)** pada tahap modal Rp 0 hingga skala menengah sangat tepat karena menghilangkan beban memori dari *database server* eksternal. Namun, keterbatasan paling mendasar dari SQLite—bahkan ketika mode WAL telah diaktifkan—adalah **batasan penulis tunggal (*single-writer constraint*)**: hanya satu transaksi tulis (INSERT, UPDATE, DELETE) yang dapat di-*commit* pada satu waktu karena SQLite mengunci seluruh struktur tabel/file saat menulis. Ketika ekosistem *omnichannel* (Telegram, WhatsApp, dan Facebook Messenger) menerima lonjakan trafik viral secara serentak—misalnya saat video promosi di Facebook memicu ratusan komentar "Mau coba" per menit yang mengaktifkan *Auto-Reply Comment*, pembuatan profil pengguna baru, klaim kredit harian, pemotongan *escrow* (atomic_escrow_charge), serta *polling* rekonsiliasi Job_ID—puluhan korutin aiosqlite yang memanggil db.commit() secara bersamaan akan saling memblokir dan memicu kegagalan sqlite3.OperationalError: database is locked (SQLITE_BUSY).
Untuk mempertahankan biaya infrastruktur Rp 0 sekaligus mengeliminasi kegagalan SQLITE_BUSY, lapisan akses SQLite wajib direfaktor melalui pemisahan koneksi baca-tulis dan optimasi pragma tingkat kernel. Pertama, saat inisialisasi koneksi aiosqlite, sistem harus mengeksekusi serangkaian instruksi PRAGMA journal_mode=WAL; PRAGMA synchronous=NORMAL; PRAGMA busy_timeout=10000; PRAGMA temp_store=MEMORY; PRAGMA mmap_size=268435456; agar operasi baca/tulis memanfaatkan *memory-mapped I/O* sebesar 256 MB di RAM dan menunggu antrean kunci hingga 10.000 milidetik sebelum melempar kesalahan. Kedua, arsitektur koneksi harus dibagi menjadi **Read Connection Pool** (4 koneksi aiosqlite dengan parameter URI ?mode=ro yang melayani *AI Support & Triage Agent*, pengecekan saldo /start, dan pembacaan status secara paralel penuh tanpa hambatan kunci) dan **Dedicated Single-Writer Coroutine** (1 koneksi tulis eksklusif yang menarik mutasi transaksi dari asyncio.Queue internal dan melakukan *batch commit* setiap 10 milidetik).

Implementasi Semantic Caching & Routing Berbasis Vektor (Valkey / RedisVL)

Dalam operasional harian **bikinAI Super Intelligence**, pengguna dari segmen UMKM, kreator konten, dan afiliasi TikTok sering kali mengirimkan instruksi yang secara semantik identik meskipun kalimatnya berbeda—seperti *"Buatkan kampanye promo diskon baju lebaran 50%"* dan *"Tolong bikin iklan promo gamis lebaran diskon setengah harga"*. Pada arsitektur saat ini, kedua permintaan tersebut dianggap sebagai kueri baru yang terpisah, sehingga masing-masing memicu pemanggilan API melalui 9Router, mengeksekusi penalaran *Muse Orchestrator*, dan menjalankan *Prompt Engineer Agent* dari awal. Selain itu, fitur *Auto-Intent Detection* pada Bagian 4.1 cetak biru menggunakan LLM di *Gatekeeper* untuk mengklasifikasikan niat pengguna (CHAT, GENERATE_IMAGE, GENERATE_VIDEO, ORCHESTRATE_CAMPAIGN, READ_DOCUMENT, IMAGE_TO_VIDEO) dalam waktu ~500 ms pada setiap pesan masuk.
Menambahkan lapisan **Semantic Caching & Vector Routing** menggunakan **Valkey** (varian